BULBASAUR

TEAM REFERENCE · ICPC · Edición 2026

CÓMO USAR ESTE CUADERNO

1. Copia la **plantilla base** en `main.cpp`.
2. Busca el algoritmo y copia el `struct` o sus funciones.
3. Lee los datos y llama a las funciones que necesites.
4. Imprime el resultado. Si el problema usa índices desde 1, suma 1 al imprimir.

COMPILAR Y EJECUTAR

```
g++ -O2 -std=c++17 prog.cpp -o prog
./prog < entrada.txt > salida.txt
```

La primera línea compila; la segunda ejecuta. Si no usas archivos, ejecuta solamente `./prog`. Las tablas indican qué hace cada función y su complejidad.

REGLAS RÁPIDAS

- Todos los índices empiezan en 0.
- Usa `long long` para contar.
- Prueba el caso mínimo antes de dudar del algoritmo.
- Revisa la lectura antes de cambiar el código.

ÍNDICE POR PROBLEMA

CAPÍTULO 1 · STRINGS p. 1
Armar y ejecutar un programa p. 1
Leer y operar strings p. 1
¿Aparece un patrón? (KMP) p. 1
Prefijos repetidos / patrón (Z) p. 2
¿Palíndromos? (Manacher) p. 2
Comparar substrings (Hash) p. 2
Distintas, ocurrencias, LCS (SAM) p. 3

CAPÍTULO 2 · JUEGOS p. 4
Ganar un juego imparcial (Grundy) p. 4
Nim y misère Nim p. 4
Dos pilas (Wythoff) p. 5
¿Gana, pierde o empata? (ciclos) p. 5

CAPÍTULO 3 · FLUJO Y MATCHING p. 5
Flujo máximo y corte mínimo (Dinic) p. 5
Matching bipartito y cover (HK) p. 6
Matching general (Blossom) p. 7

1 Strings

Un **string** es una palabra o secuencia de caracteres. En ICPC casi siempre se lee como una palabra sin espacios con `cin >> s`; si la entrada trae espacios, se lee con `getline`. Los algoritmos de este capítulo responden preguntas sobre substrings, prefijos, sufijos, palíndromos y repeticiones.

1.1 Plantilla base

Todo programa empieza así. Cambia solamente la lectura, la llamada a la receta y la impresión.

```cpp
// PLANTILLA BASE. Copia este archivo y ejecutalo con:
// g++ -O2 -std=c++17 programa.cpp -o programa
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // 1) LEER. cin >> s lee UNA palabra (se detiene en un espacio).
    string s;
    cin >> s;

    // 2) RESOLVER. Pega aqui el struct/funcion de la receta.
    //    Luego llamalo con los datos leidos.
    int largo = (int)s.size();

    // 3) IMPRIMIR. Cambia esto por lo que pida el problema.
    cout << largo << '\n';
    return 0;
}
```

Ojo: Si hay varios casos, envuelve el cuerpo del `main` en `while (t--)`. No mezcles `cin >>` con `getline` sin consumir el salto de línea (ver la receta siguiente).

1.2 Operaciones de string

C++ ya trae lo básico. Las posiciones empiezan en 0.

Quiero...	Escribo...
leer una palabra	`cin >> s;`
leer una línea con espacios	`getline(cin, s);`
largo	`s.size()`
carácter i	`s[i]`
copiar un pedazo	`s.substr(pos, len);`
buscar un texto	`s.find(t)`
agregar al final	`s += t;`
recorrer	`for (char c : s)`
invertir	`reverse(s.begin(), s.end());`
ordenar	`sort(s.begin(), s.end());`

Leer una línea después de un número

`cin >> n` deja el salto pendiente. Consúmelo antes de `getline`:

```cpp
int n;
cin >> n;
cin.ignore(numeric_limits<streamsize>::max(), '\n');
string linea;
getline(cin, linea);  // ahora si lee la linea completa
```

Ojo: `s.find(t)` devuelve `string::npos` si no encuentra `t`. Si el problema cuenta desde 1, imprime `pos + 1`.

1.3 KMP: buscar un patrón

Encuentra todas las apariciones de un patrón, incluso solapadas. Entrada: `texto patron`. Salida: posiciones 0-based.

Función	Qué hace / devuelve	Tiempo
`prefix_function(s)`	`pi[i]`: borde más largo de `s[0..i]`	$O(n)$
`kmp_search(text, pat)`	posiciones donde aparece `pat`	$O(n+m)$

```cpp
// KMP: busca TODAS las apariciones de un patron dentro de un texto.
// Entrada: texto patron (palabras sin espacios).
// Salida: cantidad de apariciones y sus posiciones (0 = primero).
#include <bits/stdc++.h>
using namespace std;

// pi[i] = largo del prefijo mas largo de s que tambien es sufijo
// de s[0..i]. Ejemplo: "ababa" -> [0,0,1,2,3].
vector<int> prefix_function(const string &s) {
    int n = (int)s.size();
    vector<int> pi(n, 0);
    for (int i = 1; i < n; ++i) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}

// Devuelve las posiciones donde empieza el patron dentro del texto.
vector<int> kmp_search(const string &text, const string &pat) {
    if (pat.empty()) return {};
    vector<int> pi = prefix_function(pat), pos;
    int j = 0; // cuantos caracteres del patron ya coinciden
    for (int i = 0; i < (int)text.size(); ++i) {
        while (j > 0 && text[i] != pat[j]) j = pi[j - 1];
        if (text[i] == pat[j]) ++j;
        if (j == (int)pat.size()) {
            pos.push_back(i - (int)pat.size() + 1);
            j = pi[j - 1]; // permite apariciones solapadas
        }
    }
    return pos;
}

int main() {
    string text, pat;
    cin >> text >> pat;

    vector<int> pos = kmp_search(text, pat);
    cout << pos.size() << '\n';
    for (int i = 0; i < (int)pos.size(); ++i) {
        if (i) cout << ' ';
```

```
        cout << pos[i];
    }
    cout << '\n';
    return 0;
}
```

El valor `pi[m-1]` es el borde más largo del patrón; el período más pequeño es `m - pi[m-1]`. Si ese período divide a `m`, el patrón es repetición de un bloque.

1.4 Z-function: prefijos repetidos

`z[i]` es el largo del prefijo más largo de `s` que coincide con `s[i..]`. Para buscar un patrón se usa `t = patron + '#' + texto`.

Función	Qué hace / devuelve	Tiempo
`z_function(s)`	arreglo `z` del string `s`	$O(n)$
patrón con `pat + '#' + text`	posiciones donde `z[i] == m`	$O(n+m)$

```
// Z-FUNCTION: z[i] = largo del prefijo mas largo de s que
// coincide con s[i..]. Sirve para buscar patrones y para
// comparar prefijos/sufijos.
#include <bits/stdc++.h>
using namespace std;

vector<int> z_function(const string &s) {
    int n = (int)s.size();
    vector<int> z(n, 0);
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
        if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
    }
    return z;
}

int main() {
    string text, pat;
    cin >> text >> pat;
    if (pat.empty()) return 0;

    // '#' debe ser un caracter que NO aparezca en text ni en pat.
    string t = pat + "#" + text;
    vector<int> z = z_function(t);

    vector<int> pos;
    int start = (int)pat.size() + 1;
    for (int i = start; i < (int)t.size(); ++i) {
        if (z[i] == (int)pat.size()) pos.push_back(i - start);
    }

    cout << pos.size() << '\n';
    for (int i = 0; i < (int)pos.size(); ++i) {
        if (i) cout << ' ';
        cout << pos[i];
    }
    cout << '\n';
    return 0;
}
```

El separador `'#'` debe ser un carácter que no aparezca en las strings. Si puede aparecer cualquiera, usa KMP.

1.5 Manacher: palíndromos

Todo sobre palíndromos en tiempo lineal. Entrada: una palabra sin espacios.

Función	Qué hace / devuelve	Tiempo
`Manacher(s)`	calcula los radios `d1` y `d2`	$O(n)$
`longest()`	{inicio, largo} del palíndromo más largo	$O(n)$
`count_all()`	palíndromos contando repetidos	$O(n)$

```
// MANACHER: todos los palindromos en O(n).
// d1[i] = radio impar: s[i-k..i+k] es palindromo.
// d2[i] = radio par:   s[i-k..i+k-1] es palindromo.
#include <bits/stdc++.h>
using namespace std;

struct Manacher {
    vector<int> d1, d2;

    Manacher(const string &s) {
        int n = (int)s.size();
        d1.assign(n, 0);
        d2.assign(n, 0);

        // Palindromos de largo impar, centro en i.
        for (int i = 0, l = 0, r = -1; i < n; ++i) {
            int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
            while (i - k >= 0 && i + k < n &&
                   s[i - k] == s[i + k]) ++k;
            d1[i] = k--;
            if (i + k > r) { l = i - k; r = i + k; }
        }

        // Palindromos de largo par, centro entre i-1 e i.
        for (int i = 0, l = 0, r = -1; i < n; ++i) {
            int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
            while (i - k - 1 >= 0 && i + k < n &&
                   s[i - k - 1] == s[i + k]) ++k;
            d2[i] = k--;
            if (i + k > r) { l = i - k - 1; r = i + k; }
        }
    }

    // Cantidad de substrings palindromas (contando repetidas).
    long long count_all() const {
        long long ans = 0;
        for (int i = 0; i < (int)d1.size(); ++i) {
            ans += d1[i] + d2[i];
        }
        return ans;
    }

    // {inicio, largo} del palindromo mas largo.
    pair<int, int> longest() const {
        int best = 0, start = 0;
        for (int i = 0; i < (int)d1.size(); ++i) {
            int len = 2 * d1[i] - 1;
            if (len > best) { best = len; start = i - d1[i] + 1; }
            len = 2 * d2[i];
            if (len > best) { best = len; start = i - d2[i]; }
        }
        return {start, best};
    }
};

int main() {
    string s;
    cin >> s;
    Manacher m(s);
    pair<int, int> p = m.longest();
    cout << "Palindromo mas largo: "
         << s.substr(p.first, p.second) << '\n';
    cout << "Su largo: " << p.second << '\n';
    cout << "Palindromos contando repetidos: "
         << m.count_all() << '\n';
    return 0;
}
```

`d1[i]` cubre palíndromos impares centrados en `i`; `d2[i]` cubre los pares centrados entre `i-1` e `i`. Si piden palíndromos **distintos**, hay que eliminar duplicados.

1.6 Hash doble: comparar substrings

Compara dos pedazos en O(1) y calcula LCP en O(log n). Los rangos son `[l, r)`: incluyen `l`, no incluyen `r`.

Función	Qué hace / devuelve	Tiempo
`StringHash(s)`	preprocesa `s`	$O(n)$
`get(l, r)`	hash de `s[l..r)`	$O(1)$
`equal(l1, r1, l2, r2)`	¿son iguales los dos rangos?	$O(1)$
`lcp(i, j)`	prefijo común de los sufijos `i`, `j`	$O(\log n)$

```
// HASH DOBLE: compara substrings en O(1) y calcula LCP en O(log n).
// Es probabilistico: con dos modulos, el riesgo de colision
// es minimo. Entrada: s, q consultas l1 r1 l2 r2.
// Los rangos son [l, r), con 0 <= l < r <= n.
// Salida: SI si las dos substrings son iguales, NO si no.
#include <bits/stdc++.h>
using namespace std;

struct StringHash {
    static const long long M1 = 1000000007LL;
    static const long long M2 = 1000000009LL;
    static const long long B  = 911382323LL;

    int n;
    vector<long long> p1, p2, h1, h2;

    StringHash(const string &s)
        : n((int)s.size()), p1(n + 1, 1), p2(n + 1, 1),
          h1(n + 1, 0), h2(n + 1, 0) {
        for (int i = 0; i < n; ++i) {
            long long c = (unsigned char)s[i] + 1;
            p1[i + 1] = p1[i] * B % M1;
            p2[i + 1] = p2[i] * B % M2;
            h1[i + 1] = (h1[i] * B + c) % M1;
            h2[i + 1] = (h2[i] * B + c) % M2;
        }
    }

    // Hash de s[l..r). El rango incluye l y NO incluye r.
    pair<long long, long long> get(int l, int r) const {
        long long a = (h1[r] - h1[l] * p1[r - l]) % M1;
        long long b = (h2[r] - h2[l] * p2[r - l]) % M2;
        if (a < 0) a += M1;
        if (b < 0) b += M2;
        return {a, b};
    }

    bool equal(int l1, int r1, int l2, int r2) const {
        if (r1 - l1 != r2 - l2) return false;
        return get(l1, r1) == get(l2, r2);
    }
```

```
    // Largo del prefijo comun mas largo de los sufijos i y j.
    int lcp(int i, int j) const {
        int low = 0, high = min(n - i, n - j);
        while (low < high) {
            int mid = (low + high + 1) / 2;
            if (get(i, i + mid) == get(j, j + mid)) low = mid;
            else high = mid - 1;
        }
        return low;
    }
};

int main() {
    string s;
    cin >> s;
    StringHash h(s);

    int q;
    cin >> q;
    while (q--) {
        int l1, r1, l2, r2;
        cin >> l1 >> r1 >> l2 >> r2;
        cout << (h.equal(l1, r1, l2, r2) ? "SI" : "NO") << '\n';
    }
    return 0;
}
```

Es probabilístico: con dos módulos el error es prácticamente nulo. Para certeza absoluta usa SAM o compara los strings.

1.7 Suffix Automaton: la estructura estrella

Resuelve casi todo sobre substrings en O(n). Usa `a–z`; las mayúsculas se convierten solas. Se construye **una vez**: `SuffixAutomaton sam(s);`.

Función	Qué hace / devuelve	Tiempo
`SuffixAutomaton(s)`	construye el autómata	$O(n)$
`distinct_substrings()`	substrings distintas	$O(n)$
`count_occurrences(p)`	veces que aparece p	$O(\|p\|)$
`longest_common_substring(t)`	substring común más larga	$O(\|t\|)$
`kth_distinct(k)`	k-ésima substring lexicográfica	$O(26\|ans\|)$
`distinct_with_min_occ(k)`	substrings con al menos k apariciones	$O(n)$

```
// SUFFIX AUTOMATON (AUTOMATA DE SUFIJOS).
// Construye en O(n) una maquina que reconoce todos los substrings.
// Usa letras 'a'..'z'; las mayusculas se pasan a minusculas.
// Entrada: s p k (k <= 10^18). Salida: varias respuestas de prueba.
#include <bits/stdc++.h>
using namespace std;

struct SuffixAutomaton {
    static const int ALPHA = 26;
    static const long long LIMIT = 4000000000000000000LL;

    struct State {
        int len, link;
        int next[ALPHA];
        State(int l = 0, int link_ = -1) : len(l), link(link_) {
            fill(next, next + ALPHA, -1);
        }
    };

    vector<State> st;       // estados del automata
    vector<int> occ;        // occ[v] = ocurrencias de sus substrings
    vector<long long> ways; // ways[v] = substrings distintos desde v
    int last;

    // Convierte 'a'..'z' y 'A'..'Z' a 0..25. Otro caracter -> -1.
    static int id(char ch) {
        if ('A' <= ch && ch <= 'Z') ch = char(ch - 'A' + 'a');
        if (ch < 'a' || ch > 'z') return -1;
        return ch - 'a';
    }

    SuffixAutomaton(const string &s = "") {
        st.reserve(2 * max(1, (int)s.size()));
        st.push_back(State()); // estado inicial
        // el estado inicial no cuenta ocurrencias
        occ.push_back(0);
        last = 0;
        for (char ch : s) extend(ch);
        compute_occurrences();
        compute_ways();
    }

    // Agrega un caracter al final del string.
    void extend(char ch) {
        int c = id(ch);
        if (c == -1) return; // ignora caracteres fuera de a-z
        int cur = (int)st.size();
        st.push_back(State(st[last].len + 1));
        occ.push_back(1); // este estado representa un prefijo real

        int p = last;
        while (p != -1 && st[p].next[c] == -1) {
            st[p].next[c] = cur;
            p = st[p].link;
        }
        if (p == -1) {
            st[cur].link = 0;
        } else {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len) {
                st[cur].link = q;
            } else {
                int clone = (int)st.size();
                st.push_back(st[q]); // copia transiciones y link
                st[clone].len = st[p].len + 1;
                occ.push_back(0); // un clon no es un prefijo real
                while (p != -1 && st[p].next[c] == q) {
                    st[p].next[c] = clone;
                    p = st[p].link;
                }
                st[q].link = st[cur].link = clone;
            }
        }
        last = cur;
    }

    // Ordena los estados por len de menor a mayor.
    vector<int> order_by_len() const {
        int n = (int)st.size();
        int max_len = st[last].len;
        vector<int> cnt(max_len + 1, 0), order(n);
        for (int i = 0; i < n; ++i) ++cnt[st[i].len];
        for (int i = 1; i <= max_len; ++i) cnt[i] += cnt[i - 1];
        for (int i = n - 1; i >= 0; --i) {
            order[--cnt[st[i].len]] = i;
        }
        return order;
    }

    // Propaga las ocurrencias por los suffix links.
    void compute_occurrences() {
        vector<int> order = order_by_len();
        for (int i = (int)order.size() - 1; i > 0; --i) {
            int v = order[i];
            if (st[v].link != -1) occ[st[v].link] += occ[v];
        }
    }

    // Cuenta los caminos (substrings distintos) desde cada estado.
    void compute_ways() {
        int n = (int)st.size();
        ways.assign(n, 0);
        vector<int> order = order_by_len();
        for (int i = n - 1; i >= 0; --i) {
            int v = order[i];
            long long total = 0;
            for (int c = 0; c < ALPHA; ++c) {
                int u = st[v].next[c];
                if (u == -1) continue;
                // la arista sola + todas sus extensiones
                total += 1 + ways[u];
                if (total > LIMIT) { total = LIMIT; break; }
            }
            ways[v] = total;
        }
    }

    // Cantidad de substrings distintos (sin contar el string vacio).
    long long distinct_substrings() const {
        long long ans = 0;
        for (int v = 1; v < (int)st.size(); ++v) {
            ans += st[v].len - st[st[v].link].len;
        }
        return ans;
    }

    // Cuantas veces aparece p en s. Devuelve 0 si no aparece.
    int count_occurrences(const string &p) const {
        if (p.empty()) return 0;
        int v = 0;
        for (char ch : p) {
            int c = id(ch);
            if (c == -1 || st[v].next[c] == -1) return 0;
            v = st[v].next[c];
        }
        return occ[v];
    }

    // Largo del substring comun mas largo entre s y t.
    int longest_common_substring(const string &t) const {
        int v = 0, len = 0, best = 0;
        for (char ch : t) {
            int c = id(ch);
            if (c == -1) { v = 0; len = 0; continue; }
            while (v != -1 && st[v].next[c] == -1) v = st[v].link;
            if (v == -1) { v = 0; len = 0; continue; }
            len = min(len, st[v].len) + 1;
            v = st[v].next[c];
            best = max(best, len);
        }
        return best;
    }

    // k-esima substring distinta en orden lexicografico
    // (k empieza en 1).
    // Si no existe, devuelve "".
    string kth_distinct(long long k) const {
        if (k <= 0 || k > ways[0]) return "";
        string ans;
        int v = 0;
        while (true) {
            for (int c = 0; c < ALPHA; ++c) {
                int u = st[v].next[c];
                if (u == -1) continue;
                // substrings que empiezan con esta letra
                long long cnt = 1 + ways[u];
                if (k > cnt) { k -= cnt; continue; }
                ans.push_back(char('a' + c));
                --k;
                if (k == 0) return ans;
                v = u;
                break;
```

```
            }
        }
    }

    // Cantidad de substrings distintos que aparecen al menos k veces.
    long long distinct_with_min_occ(int k) const {
        if (k <= 0) return distinct_substrings();
        long long ans = 0;
        for (int v = 1; v < (int)st.size(); ++v) {
            if (occ[v] >= k) ans += st[v].len - st[st[v].link].len;
        }
        return ans;
    }
};

int main() {
    string s, p;
    long long k;
    cin >> s >> p >> k;

    SuffixAutomaton sam(s);
    cout << "Distintas: " << sam.distinct_substrings() << '\n';
    cout << "Ocurrencias de p: " << sam.count_occurrences(p) << '\n';
    cout << "LCS con p: " << sam.longest_common_substring(p) << '\n';
    cout << "k-esima: " << sam.kth_distinct(k) << '\n';
    cout << "Con >=2 ocurrencias: "
         << sam.distinct_with_min_occ(2) << '\n';
    return 0;
}
```

Ojo: Memoria O($26n$). `kth_distinct` devuelve `""` si `k` no existe (vale hasta `10^18`). No agregues caracteres después de construir sin recalcular `occ`.

2 Teoría de juegos

En un juego normal, el jugador que no puede mover pierde. Las funciones de este capítulo calculan si gana el jugador que tiene el turno.

2.1 Grundy y Sprague-Grundy

Un juego imparcial se parte en componentes independientes. El SG de una posición es el `mex` de los SG de sus movimientos. Si el XOR de los SG de las componentes es distinto de 0, gana el primero.

Función	Qué hace / devuelve	Tiempo
`mex(v)`	menor entero no negativo que no está en v	$O(k)$
`subtraction_grundy`	SG de una pila de 0 a n	$O(nm)$
`GrundyDAG::calc()`	SG de un DAG (o {} si hay ciclo)	$O(V+E)$

```
// GRUNDY / SPRAUGE-GRUNDY.
// Juego imparcial sin ciclos: gana quien no puede mover pierde.
// El SG de una posicion es el mex de los SG de sus sucesoras.
// Varias componentes independientes se combinan con XOR de sus SG.
#include <bits/stdc++.h>
using namespace std;

// Menor entero no negativo que no esta en v.
int mex(const vector<int>& v) {
    vector<char> seen(v.size() + 1, 0);
    for (int x : v) {
        if (x >= 0 && x <= (int)v.size()) seen[x] = 1;
    }
    int g = 0;
    while (g < (int)seen.size() && seen[g]) ++g;
    return g;
}

// Grafo dirigido sin ciclos. g[v] = estados a los que se puede mover.
struct GrundyDAG {
    int n;
    vector<vector<int>> g;
    GrundyDAG(int n_) : n(n_), g(n_) {}
    void add_edge(int u, int v) { g[u].push_back(v); }

    // Devuelve sg[v] para cada estado. Si hay un ciclo, devuelve {}.
    vector<int> calc() const {
        vector<int> indeg(n, 0), order;
        for (int u = 0; u < n; ++u) {
            for (int v : g[u]) ++indeg[v];
        }
        queue<int> q;
        for (int i = 0; i < n; ++i) {
            if (indeg[i] == 0) q.push(i);
        }
        while (!q.empty()) {
            int u = q.front(); q.pop();
            order.push_back(u);
            for (int v : g[u]) {
                if (--indeg[v] == 0) q.push(v);
            }
        }
        if ((int)order.size() != n) return {};

        vector<int> sg(n, 0), mark(n + 1, 0);
        int stamp = 0;
        for (int i = n - 1; i >= 0; --i) {
            int u = order[i];
            ++stamp;
            for (int v : g[u]) {
                if (sg[v] <= n) mark[sg[v]] = stamp;
            }
            int x = 0;
            while (mark[x] == stamp) ++x;
            sg[u] = x;
        }
        return sg;
    }
};

// Juego de una pila: desde x se puede quitar cualquier move <= x.
vector<int> subtraction_grundy(int max_n, const vector<int>& moves) {
    vector<int> sg(max_n + 1, 0), mark(max_n + 1, 0);
    for (int x = 1; x <= max_n; ++x) {
        for (int mv : moves) {
            if (mv <= x) mark[sg[x - mv]] = x;
        }
        int g = 0;
        while (mark[g] == x) ++g;
        sg[x] = g;
    }
    return sg;
}

int main() {
    int n, m;
    cin >> n >> m;  // pila inicial y cantidad de movimientos
    vector<int> moves(m);
    for (int &x : moves) cin >> x;

    vector<int> sg = subtraction_grundy(n, moves);
    cout << "SG(" << n << ") = " << sg[n] << '\n';
    cout << (sg[n] ? "Gana el primero" : "Gana el segundo") << '\n';
    return 0;
}
```

Ojo: Para varias pilas o componentes, combina con XOR: `g = g1 ^ g2 ^ ...`. Si `g != 0` gana el primero. Un estado terminal tiene SG 0.

2.2 Nim y misère Nim

Varias pilas; en cada turno se quita cualquier cantidad positiva de una sola pila. En misère, el que toma la última piedra pierde.

Función	Qué hace / devuelve	Tiempo
`nim_win(piles)`	¿gana el primero?	$O(n)$
`misere_nim_win(piles)`	versión misère	$O(n)$
`nim_move(piles)`	{pila, nuevo tamaño} que deja XOR 0	$O(n)$

```
// NIM: varias pilas, se quita cualquier cantidad de una sola pila.
// Con una sola pila el que toma la ultima gana; con varias, gana
// quien deja XOR = 0.
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// Juego normal: el que no puede mover pierde.
bool nim_win(const vector<int>& piles) {
    int x = 0;
    for (int p : piles) x ^= p;
    return x != 0;
}

// Misere: el que toma la ultima piedra pierde.
bool misere_nim_win(const vector<int>& piles) {
    int x = 0;
    bool all_one = true;
    for (int p : piles) {
        x ^= p;
        if (p > 1) all_one = false;
    }
    if (all_one) return x == 0;
    return x != 0;
}

// Devuelve {indice, nuevo_tamano} para dejar XOR = 0.
// Si no hay jugada ganadora, devuelve {-1, -1}.
pair<int, int> nim_move(const vector<int>& piles) {
    int x = 0;
    for (int p : piles) x ^= p;
    if (x == 0) return {-1, -1};
    for (int i = 0; i < (int)piles.size(); ++i) {
        int target = piles[i] ^ x;
        if (target < piles[i]) return {i, target};
    }
    return {-1, -1};
}

int main() {
    int n;
    cin >> n;
    vector<int> piles(n);
    for (int &p : piles) cin >> p;

    cout << "Normal: " << (nim_win(piles) ? "gana 1" : "gana 2") << '\n';
    cout << "Misere: "
         << (misere_nim_win(piles) ? "gana 1" : "gana 2") << '\n';

    pair<int, int> mv = nim_move(piles);
    if (mv.first == -1) cout << "No hay jugada ganadora\n";
    else {
        cout << "Mueve pila " << mv.first
             << " a " << mv.second << '\n';
    }
```

```
    return 0;
}
```

Ojo: En misère, si todas las pilas son 0 o 1, gana el primero cuando la cantidad de pilas es par. En otro caso se juega como Nim normal.

2.3 Wythoff: dos pilas

Se puede quitar cualquier cantidad de una pila, o la misma cantidad de las dos. Las posiciones perdedoras son $(\lfloor k\varphi \rfloor, \lfloor k\varphi \rfloor + k)$.

Función	Qué hace / devuelve	Tiempo
`floor_phi(d)`	$\lfloor d\varphi \rfloor$ sin error de coma flotante	$O(\log d)$
`wythoff_win(a, b)`	¿gana el primero?	$O(\log d)$

```
// WYTHOFF: dos pilas. Se puede quitar cualquier cantidad de una pila
// o la misma cantidad de ambas. Las posiciones perdedoras son
// (floor(k*phi), floor(k*phi) + k), con phi = (1 + sqrt(5)) / 2.
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// floor(d * phi), calculado sin errores de punto flotante.
ll floor_phi(ll d) {
    ll low = 0, high = 2 * d + 1;  // phi < 2
    while (low + 1 < high) {
        ll mid = (low + high) / 2;
        __int128 a = (__int128)2 * mid - d;
        bool le = ((__int128)2 * mid <= d) ||
                  (a * a <= (__int128)5 * d * d);
        if (le) low = mid;
        else high = mid;
    }
    return low;
}

// true si el primero gana desde (a, b).
bool wythoff_win(ll a, ll b) {
    if (a > b) swap(a, b);
    ll d = b - a;
    return a != floor_phi(d);
}

int main() {
    ll a, b;
    cin >> a >> b;
    cout << (wythoff_win(a, b) ? "Gana el primero" : "Gana el segundo");
    cout << '\n';
    return 0;
}
```

Ojo: El código usa `__int128` y búsqueda binaria para no depender de la precisión de `double`. Si `a == floor\_phi(b-a)` (con `a <= b`), la posición es perdedora.

2.4 Juegos con ciclos: análisis retrógrado

Un estado sin movimientos pierde. Si puedo mover a un estado perdedor, gano; si todos mis movimientos llevan a estados ganadores, pierdo. Lo que queda es empate.

Función	Qué hace / devuelve	Tiempo
`solve_cyclic_game(n, g)`	$1 =$ gana, $-1 =$ pierde, $0 =$ empate	$O(V + E)$

```
// JUEGOS CON CICLOS: analisis retrogrado.
// Los estados sin movimientos son perdedores. Un estado es ganador
// si puede mover a uno perdedor; es perdedor si todos sus movimientos
// llevan a estados ganadores. Lo que queda es empate.
#include <bits/stdc++.h>
using namespace std;

// Devuelve 1 = gana, -1 = pierde, 0 = empate.
vector<int> solve_cyclic_game(int n, const vector<vector<int>>& g) {
    vector<int> out(n, 0), state(n, 0);
    vector<vector<int>> rev(n);
    for (int u = 0; u < n; ++u) {
        for (int v : g[u]) {
            ++out[u];
            rev[v].push_back(u);
        }
    }

    queue<int> q;
    for (int v = 0; v < n; ++v) {
        if (out[v] == 0) {
            state[v] = -1;  // sin movimientos: pierde
            q.push(v);
        }
    }

    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (int u : rev[v]) {
            if (state[u] != 0) continue;
            if (state[v] == -1) {
                state[u] = 1;  // puede mover a un perdedor
                q.push(u);
            } else {
                if (--out[u] == 0) {
                    state[u] = -1;  // todos sus movimientos ganan
                    q.push(u);
                }
            }
        }
    }
    return state;  // los 0 que quedan son empates
}

int main() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> g(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        g[u].push_back(v);
    }

    vector<int> st = solve_cyclic_game(n, g);
    for (int i = 0; i < n; ++i) {
        cout << i << ": ";
        if (st[i] == 1) cout << "GANA\n";
        else if (st[i] == -1) cout << "PIERDE\n";
        else cout << "EMPATE\n";
    }
    return 0;
}
```

Ojo: Sirve para juegos con tableros y movimientos que pueden repetir estados. Si el grafo es un DAG, no hay empates y puedes usar Grundy en vez de esta función.

3 Flujo y matching

Dinic resuelve flujo con capacidades. Para matching bipartito usa Hopcroft-Karp; para grafos generales, Blossom.

3.1 Dinic: flujo máximo y corte mínimo

Grafo dirigido con capacidades `long long`. Devuelve el flujo máximo. Después del flujo, `min_cut(s)` marca el lado S del corte mínimo.

Función	Qué hace / devuelve	Tiempo
`Dinic(n)`	crea un grafo con `n` nodos	$O(n)$
`add_edge(u, v, cap)`	agrega arista dirigida con capacidad	$O(1)$
`max_flow(s, t)`	valor del flujo máximo	$O(V^2E)$
`min_cut(s)`	lado S después de saturar	$O(V + E)$
`cut_edges(s)`	aristas originales que cruzan el corte	$O(V + E)$

```
// DINIC: flujo maximo y corte minimo en un grafo dirigido.
// Las capacidades son long long. Para flujo con capacidades unitarias
// en redes bipartitas, Hopcroft-Karp suele ser mas simple y rapido.
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct Dinic {
    struct Edge {
        int to, rev;
        ll cap;
    };
    struct Raw {
        int u, v;
        ll cap;
    };

    int n;
    vector<vector<Edge>> g;
    vector<Raw> raw;
    vector<int> level, it;

    Dinic(int n_) : n(n_), g(n_), level(n_), it(n_) {}

    void add_edge(int u, int v, ll cap) {
        raw.push_back({u, v, cap});
        Edge a{v, (int)g[v].size(), cap};
        Edge b{u, (int)g[u].size(), 0};
        g[u].push_back(a);
        g[v].push_back(b);
    }

    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (const Edge &e : g[v]) {
                if (e.cap > 0 && level[e.to] == -1) {
                    level[e.to] = level[v] + 1;
                    q.push(e.to);
                }
            }
        }
```

```
        return level[t] != -1;
    }

    ll dfs(int v, int t, ll pushed) {
        if (v == t) return pushed;
        for (int &i = it[v]; i < (int)g[v].size(); ++i) {
            Edge &e = g[v][i];
            if (e.cap == 0 || level[e.to] != level[v] + 1) continue;
            ll got = dfs(e.to, t, min(pushed, e.cap));
            if (got == 0) continue;
            e.cap -= got;
            g[e.to][e.rev].cap += got;
            return got;
        }
        return 0;
    }

    ll max_flow(int s, int t) {
        ll flow = 0;
        const ll INF = (1LL << 60);
        while (bfs(s, t)) {
            fill(it.begin(), it.end(), 0);
            while (ll got = dfs(s, t, INF)) flow += got;
        }
        return flow;
    }

    // Lado S del corte minimo despues de calcular max_flow.
    vector<char> min_cut(int s) const {
        vector<char> vis(n, 0);
        queue<int> q;
        vis[s] = 1;
        q.push(s);
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (const Edge &e : g[v]) {
                if (e.cap > 0 && !vis[e.to]) {
                    vis[e.to] = 1;
                    q.push(e.to);
                }
            }
        }
        return vis;
    }

    // Aristas originales que cruzan el corte S -> T.
    vector<pair<int, int>> cut_edges(int s) const {
        vector<char> side = min_cut(s);
        vector<pair<int, int>> ans;
        for (const Raw &e : raw) {
            if (side[e.u] && !side[e.v]) ans.push_back({e.u, e.v});
        }
        return ans;
    }
};

int main() {
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    Dinic din(n);
    while (m--) {
        int u, v;
        ll c;
        cin >> u >> v >> c;
        din.add_edge(u, v, c);
    }

    cout << "Flujo maximo: " << din.max_flow(s, t) << '\n';
    vector<pair<int, int>> cut = din.cut_edges(s);
    cout << "Aristas del corte: " << cut.size() << '\n';
    for (auto [u, v] : cut) cout << u << " -> " << v << '\n';
    return 0;
}
```

Ojo: Para capacidades unitarias en una red bipartita, Hopcroft-Karp es más simple. Para vértices con capacidad, divide cada vértice en entrada y salida con una arista de capacidad.

3.2 Hopcroft-Karp: matching bipartito

Matching máximo en un grafo bipartito. Izquierda: `0..n-1`; derecha: `0..m-1`. El cover usa `0..n-1` para la izquierda y `n..n+m-1` para la derecha.

Función	Qué hace / devuelve	Tiempo
`HopcroftKarp(n, m)`	crea las dos particiones	$O(n+m)$
`add_edge(u, v)`	arista izquierda u – derecha v	$O(1)$
`max_matching()`	tamaño del matching máximo	$O(E\sqrt{V})$
`min_vertex_cover()`	cover mínimo (Kőnig)	$O(V+E)$
`max_independent_set()`	conjunto independiente máximo	$O(V+E)$
`min_path_cover_dag`	cobertura mínima de caminos en un DAG	$O(E\sqrt{V})$

```
// HOPCROFT-KARP: matching maximo en grafo bipartito.
// Izquierda: 0..n-1. Derecha: 0..m-1 (se guardan como n+v si hace falta).
#include <bits/stdc++.h>
using namespace std;

struct HopcroftKarp {
    int n, m;
    vector<vector<int>> adj;
    vector<int> pairU, pairV, dist;

    HopcroftKarp(int n_, int m_)
        : n(n_), m(m_), adj(n_), pairU(n_, -1), pairV(m_, -1),
          dist(n_) {}

    void add_edge(int u, int v) { adj[u].push_back(v); }

    bool bfs() {
        queue<int> q;
        for (int u = 0; u < n; ++u) {
            if (pairU[u] == -1) {
                dist[u] = 0;
                q.push(u);
            } else {
                dist[u] = -1;
            }
        }
        bool found = false;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : adj[u]) {
                int u2 = pairV[v];
                if (u2 == -1) {
                    found = true;
                } else if (dist[u2] == -1) {
                    dist[u2] = dist[u] + 1;
                    q.push(u2);
                }
            }
        }
        return found;
    }

    bool dfs(int u) {
        for (int v : adj[u]) {
            int u2 = pairV[v];
            if (u2 == -1 || (dist[u2] == dist[u] + 1 && dfs(u2))) {
                pairU[u] = v;
                pairV[v] = u;
                return true;
            }
        }
        dist[u] = -1;
        return false;
    }

    int max_matching() {
        int matching = 0;
        while (bfs()) {
            for (int u = 0; u < n; ++u) {
                if (pairU[u] == -1 && dfs(u)) ++matching;
            }
        }
        return matching;
    }

    // Vertices 0..n-1 = izquierda; n..n+m-1 = derecha.
    vector<int> min_vertex_cover() const {
        vector<char> visU(n, 0), visV(m, 0);
        queue<int> q;
        for (int u = 0; u < n; ++u) {
            if (pairU[u] == -1) {
                visU[u] = 1;
                q.push(u);
            }
        }
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : adj[u]) {
                if (pairU[u] == v) continue;  // solo aristas no matching
                if (visV[v]) continue;
                visV[v] = 1;
                if (pairV[v] != -1 && !visU[pairV[v]]) {
                    visU[pairV[v]] = 1;
                    q.push(pairV[v]);
                }
            }
        }
        vector<int> cover;
        for (int u = 0; u < n; ++u) {
            if (!visU[u]) cover.push_back(u);
        }
        for (int v = 0; v < m; ++v) {
            if (visV[v]) cover.push_back(n + v);
        }
        return cover;
    }

    vector<int> max_independent_set() const {
        vector<int> cover = min_vertex_cover();
        vector<char> used(n + m, 0);
        for (int x : cover) used[x] = 1;
        vector<int> ans;
        for (int i = 0; i < n + m; ++i) {
            if (!used[i]) ans.push_back(i);
        }
        return ans;
    }
};

// Cobertura minima de caminos en un DAG: n - matching.
int min_path_cover_dag(int n, const vector<pair<int, int>>& edges) {
    HopcroftKarp hk(n, n);
    for (auto [u, v] : edges) hk.add_edge(u, v);
    return n - hk.max_matching();
}

int main() {
    int n, m, e;
    cin >> n >> m >> e;
    HopcroftKarp hk(n, m);
```

```
    while (e--) {
        int u, v;
        cin >> u >> v;
        hk.add_edge(u, v);
    }

    cout << "Matching: " << hk.max_matching() << '\n';
    vector<int> cover = hk.min_vertex_cover();
    cout << "Vertices del cover: " << cover.size() << '\n';
    for (int x : cover) cout << x << ' ';
    cout << '\n';

    vector<int> indep = hk.max_independent_set();
    cout << "Independiente: " << indep.size() << '\n';
    return 0;
}
```

Ojo: Si solo necesitas el tamaño del matching, `max_matching()` es suficiente. El cover y el independiente son complementarios: su tamaño suma `n + m`.

3.3 Blossom: matching general

Matching máximo en un grafo no bipartito (Edmonds). `add_edge` agrega la arista en ambos sentidos. Para `n` hasta unos 500.

Función	Qué hace / devuelve	Tiempo
`Blossom(n)`	crea el grafo	$O(n)$
`add_edge(u, v)`	arista no dirigida	$O(1)$
`max_matching()`	tamaño del matching máximo	$O(V^3)$
`match[i]`	pareja de i, o -1	$O(1)$

```
// BLOSSOM (Edmonds): matching maximo en grafo general no bipartito.
// add_edge agrega la arista en ambos sentidos. V ~ 500.
#include <bits/stdc++.h>
using namespace std;

struct Blossom {
    int n;
    vector<vector<int>> g;
    vector<int> match, p, base, q;
    vector<char> used, blossom;

    Blossom(int n_)
        : n(n_), g(n_), match(n_, -1), p(n_), base(n_), used(n_),
          blossom(n_) {}

    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    int lca(int a, int b) {
        vector<char> seen(n, 0);
        while (true) {
            a = base[a];
            seen[a] = 1;
            if (match[a] == -1) break;
            a = p[match[a]];
        }
        while (true) {
            b = base[b];
            if (seen[b]) return b;
            b = p[match[b]];
        }
    }

    void mark_path(int v, int b, int child) {
        while (base[v] != b) {
            blossom[base[v]] = blossom[base[match[v]]] = 1;
            p[v] = child;
            child = match[v];
            v = p[match[v]];
        }
    }

    int find_path(int root) {
        used.assign(n, 0);
        p.assign(n, -1);
        for (int i = 0; i < n; ++i) base[i] = i;
        q.clear();
        q.push_back(root);
        used[root] = 1;

        for (int qi = 0; qi < (int)q.size(); ++qi) {
            int v = q[qi];
            for (int to : g[v]) {
                if (base[v] == base[to] || match[v] == to) continue;
                if (to == root ||
                    (match[to] != -1 && p[match[to]] != -1)) {
                    int cur = lca(v, to);
                    blossom.assign(n, 0);
                    mark_path(v, cur, to);
                    mark_path(to, cur, v);
                    for (int i = 0; i < n; ++i) {
                        if (blossom[base[i]]) {
                            base[i] = cur;
                            if (!used[i]) {
                                used[i] = 1;
                                q.push_back(i);
                            }
                        }
                    }
                } else if (p[to] == -1) {
                    p[to] = v;
                    if (match[to] == -1) return to;
                    to = match[to];
                    used[to] = 1;
                    q.push_back(to);
                }
            }
        }
        return -1;
    }

    int max_matching() {
        match.assign(n, -1);
        for (int i = 0; i < n; ++i) {
            if (match[i] != -1) continue;
            int v = find_path(i);
            while (v != -1) {
                int pv = p[v], ppv = match[pv];
                match[v] = pv;
                match[pv] = v;
                v = ppv;
            }
        }
        int ans = 0;
        for (int x : match) {
            if (x != -1) ++ans;
        }
        return ans / 2;
    }
};

int main() {
    int n, m;
    cin >> n >> m;
    Blossom bl(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        bl.add_edge(u, v);
    }

    cout << "Matching: " << bl.max_matching() << '\n';
    for (int i = 0; i < n; ++i) {
        if (i < bl.match[i]) cout << i << " - " << bl.match[i] << '\n';
    }
    return 0;
}
```

Ojo: En un grafo bipartito no uses Blossom: Hopcroft-Karp es mucho más rápido. Blossom sirve para grafos generales con ciclos impares.